Agentic AI for Process Capability & Behavior Assessment: Pilot Implementation & Management Review

1. Solution Overview

The implemented Azure AI solution satisfies the pilot requirements of building a working agentic workflow that integrates multiple AI capabilities and Azure services. Specifically, this project includes:

1. A fully functional Prompt Flow / Workflow hosted in an Azure AI Hub Project.

2. End-to-end data ingestion from Azure Blob Storage using a Python data access node.

3. Parallel analytical branches for:
    a. Process capability analysis using an MCP-backed agent.
    b. Process behavior (stability) analysis using chart generation and a multimodal agent.

4. A persistent, RAG-backed Aggregator Agent hosted in an Azure AI Foundry Project that synthesizes results into a leadership-ready narrative.

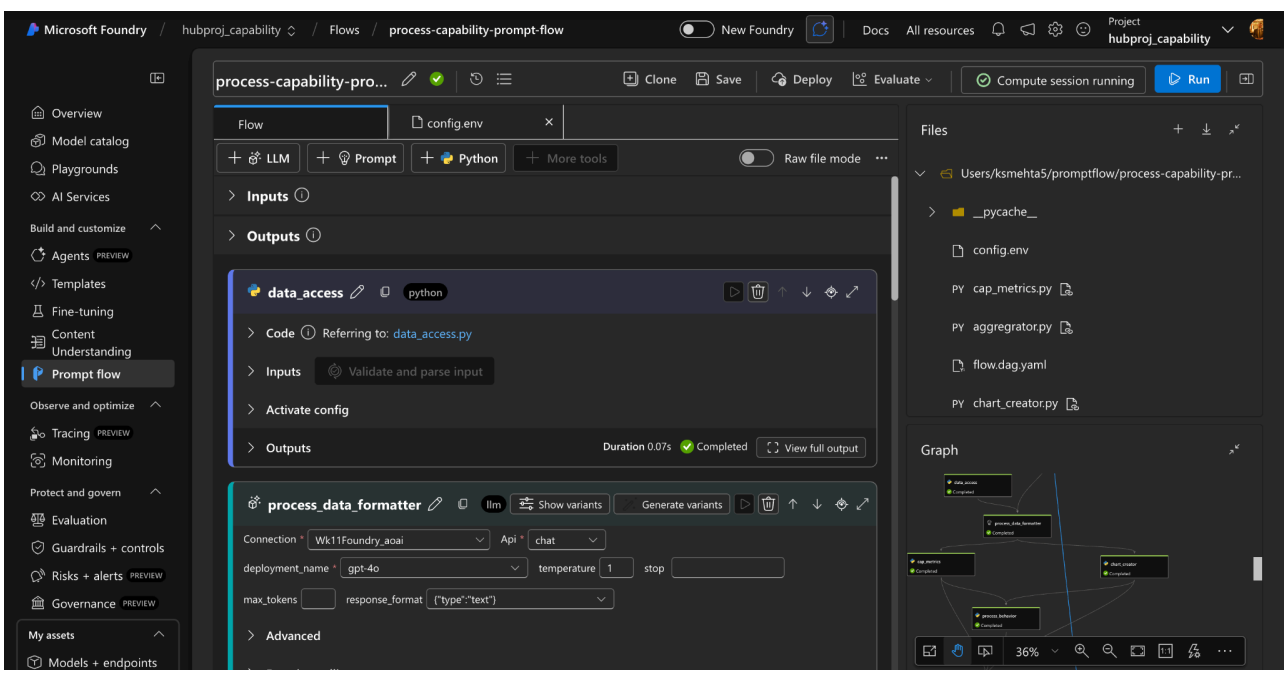

Fig. 1: Prompt Flow build interface

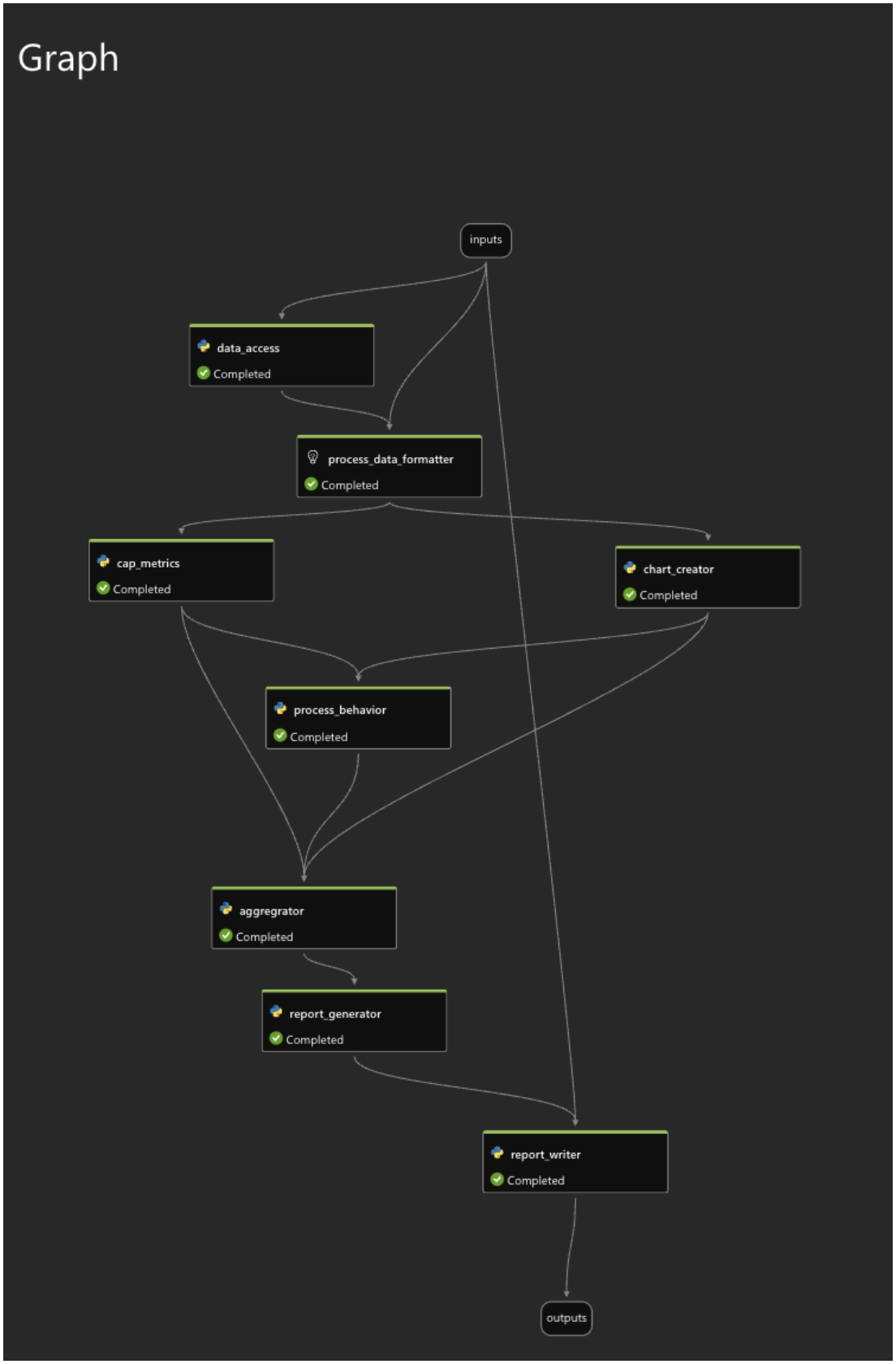

Fig. 2: Prompt Flow Graph

2. Use of Multiple Agent Patterns

This solution demonstrates all three major agent patterns emphasized in the pilot pitch:

- On-the-fly MCP agents for standardized capability calculations and chart generation.
- On-the-fly multimodal agent for visual interpretation of process behavior charts.
- Persistent RAG-backed agent for governed, standards-aligned interpretation and recommendations.

These patterns are orchestrated within a single workflow, illustrating how specialized agents can be composed to solve a real operational decision problem.

3. Node-by-Node Explanation and Alignment to the Intended Goal

3.1 Overall Intended Goal

The goal of the NRD agentic workflow is to transform raw process data into a clear, leadership-ready assessment of process stability and capability, grounded in NRD's internal standards, while reducing analysis time and interpretation inconsistency.

Each node in the workflow contributes a specific capability toward that goal, with clear separation of responsibilities between data access, analysis, interpretation, and synthesis.

3.2 data_access (Python Node)

What it does

- Connects to Azure Blob Storage using provided credentials.
- Retrieves the specified CSV file containing process measurement data.
- Returns the raw data to the workflow for downstream processing.

Why this node exists

This node decouples data access from analysis logic. Instead of embedding file paths or manual uploads, the workflow pulls data programmatically from cloud storage, mirroring how real production systems operate.

How it ties to the goal:

- Ensures the workflow operates on a single source of truth.
- Enables scalability to many processes without changing the workflow logic.
- Establishes a realistic foundation for automation and future integration.

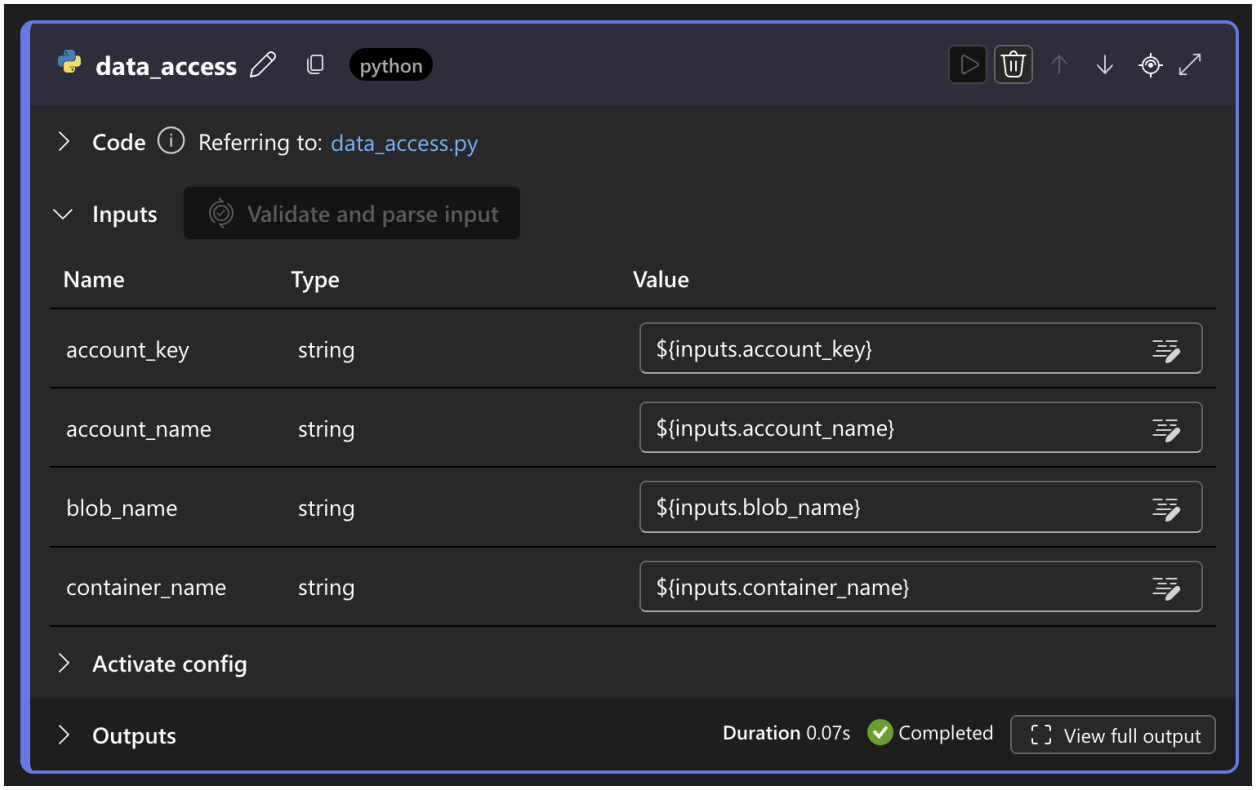

Fig. 3: data_access node config

3.3 process_data_formatter (LLM Node)

What it does

- Uses an LLM and a structured Jinja2 prompt template to:
- Interpret the raw CSV structure.
- Identify the correct measurement column.
- Align data with specification limits (LSL, Target, USL).
- Outputs a standardized data object for downstream agents.

Why this node exists

Process data often varies in column names, ordering, or formatting. Instead of writing brittle parsing code, this node leverages the LLM's reasoning to normalize the data into a consistent structure.

How it ties to the goal

- Reduces preprocessing effort and human error.
- Ensures all downstream analyses operate on clean, consistent inputs.
- Makes the workflow robust to minor data format changes.

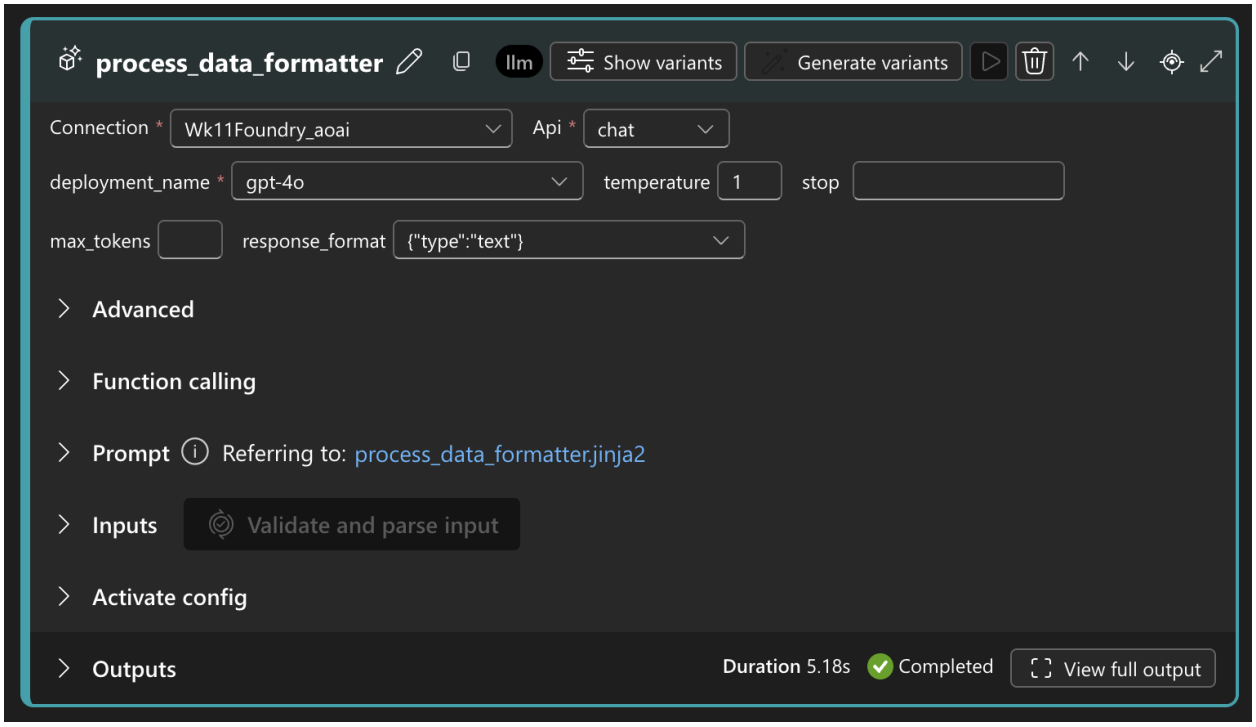

Fig. 4: process_data_formatter node config

3.4 cap_metrics (Python Node with MCP Agent)

What it does

- Creates an on-the-fly agent with a Process Capability MCP tool attached.
- Calculates standardized capability metrics (e.g., Cp, Cpk).
- Produces both numeric results and a short textual summary.

Why this node exists

Capability calculations must be consistent, auditable, and standardized. By encapsulating calculations in an MCP tool, the workflow avoids re-implementing statistics in prompts or ad hoc scripts.

How it ties to the goal

- Ensures capability metrics are computed the same way every time.
- Supports defensible, repeatable decision-making.
- Frees engineers from repetitive calculation work.

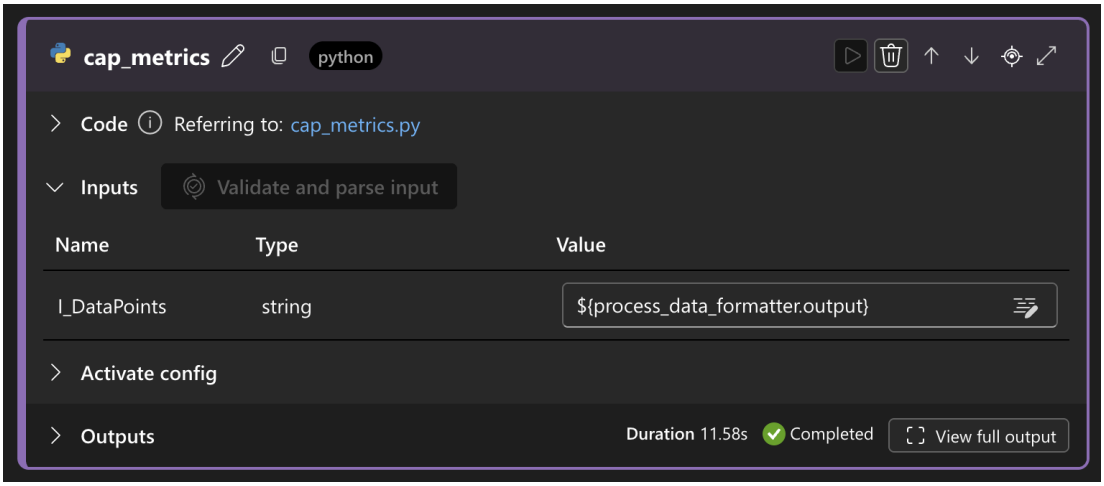

Fig. 5: cap_metrics node config

3.5 chart_creator (Python Node with MCP Agent)

What it does

- Creates an on-the-fly agent with a chart-generation MCP tool.
- Produces a process behavior chart showing data over time and behavior limits.
- Outputs a URL to the generated chart.

Why this node exists

Visualizing process behavior is a critical step in determining stability. Separating chart generation from interpretation mirrors real-world workflows where charts are created by one tool and interpreted by another.

How it ties to the goal

- Provides a visual artifact for stability assessment.
- Enables multimodal analysis in downstream nodes.
- Preserves traceability between data and interpretation.

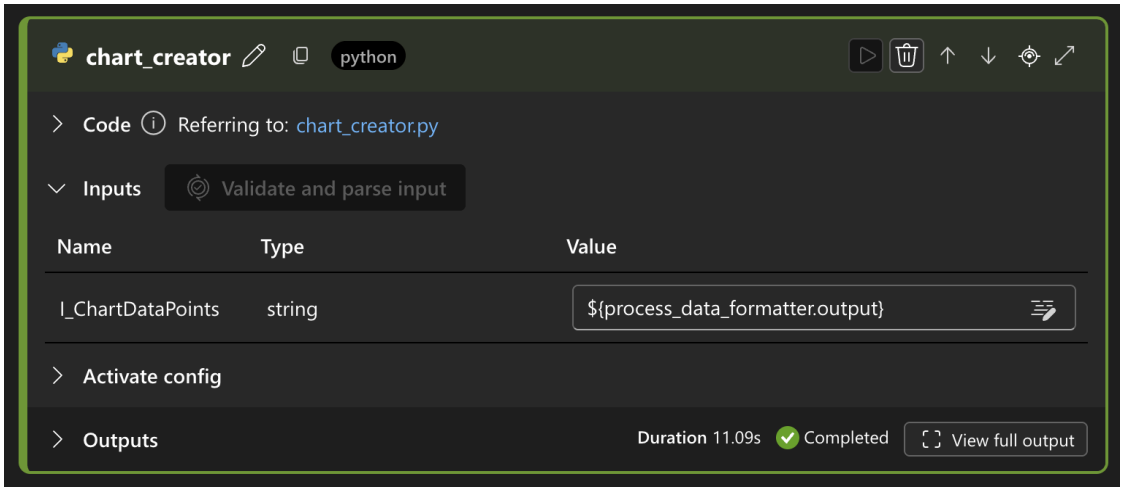

Fig. 6: chart_creator node config

3.6 process_behavior (Python Node with Multimodal Agent)

What it does

- Uses a vision-capable (multimodal) agent to “look at” the chart image.
- Applies process behavior rules to determine stability vs. instability.
- Produces a clear stability summary and implications.

Why this node exists

Stability assessment often depends on visual pattern recognition (out-of-limit points, runs, trends). This node explicitly exercises multimodal reasoning rather than relying only on numeric thresholds.

How it ties to the goal

- Enforces the critical rule that stability must be assessed before trusting capability indices.
- Mimics expert human judgment in a repeatable way.
- Prevents misleading conclusions from unstable data.

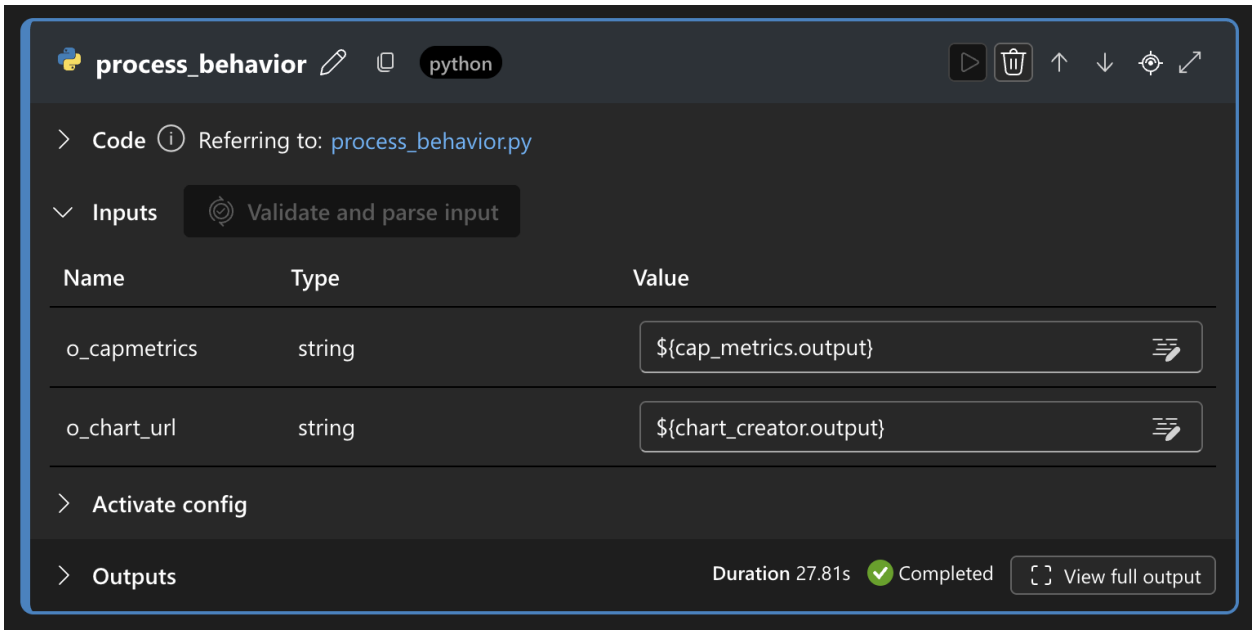

Fig. 7: process_behavior node config

3.7 aggregator (Python Node Calling Persistent RAG Agent)

What it does

- Calls a persistent Aggregator Agent hosted in Azure AI Foundry.
- Passes capability results and stability summaries.
- Uses RAG grounded in NRD’s internal capability guidance PDFs.
- Produces a unified, leadership-ready narrative.

Why this node exists

Raw metrics and charts alone are not decision-ready. This node centralizes interpretation logic in a governed, reusable “corporate expert” agent that aligns with internal standards.

How it ties to the goal

- Converts analysis into actionable insight.
- Ensures consistency across engineers and runs.
- Reduces interpretation variability and debate.

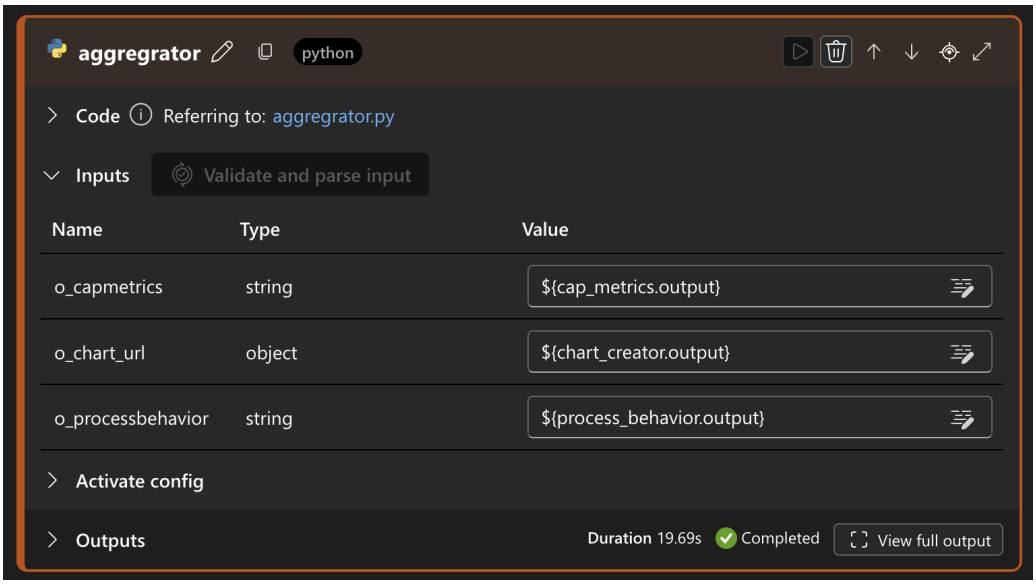

Fig. 8: aggregator node config

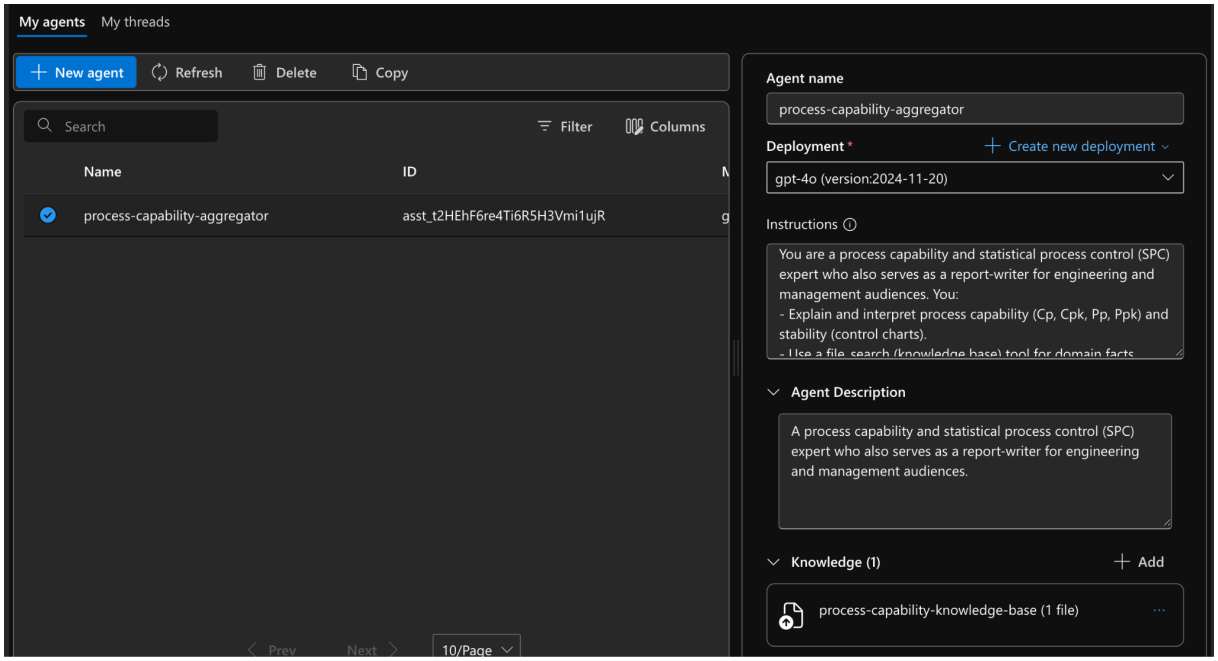

Fig. 9: process-capability-aggregator Agent build interface in Foundry project

3.8 report_generator & report_writer

What they do

- Transform the aggregated narrative into structured HTML or report formats.
- Enable publishing or distribution of results.

Why these nodes exist

They demonstrate how analytical outputs can be packaged for consumption beyond the Prompt Flow UI, without altering the core logic.

How they tie to the goal

- Show readiness for operationalization.
- Support executive reporting and downstream automation.
- Reinforce separation between analysis and presentation.

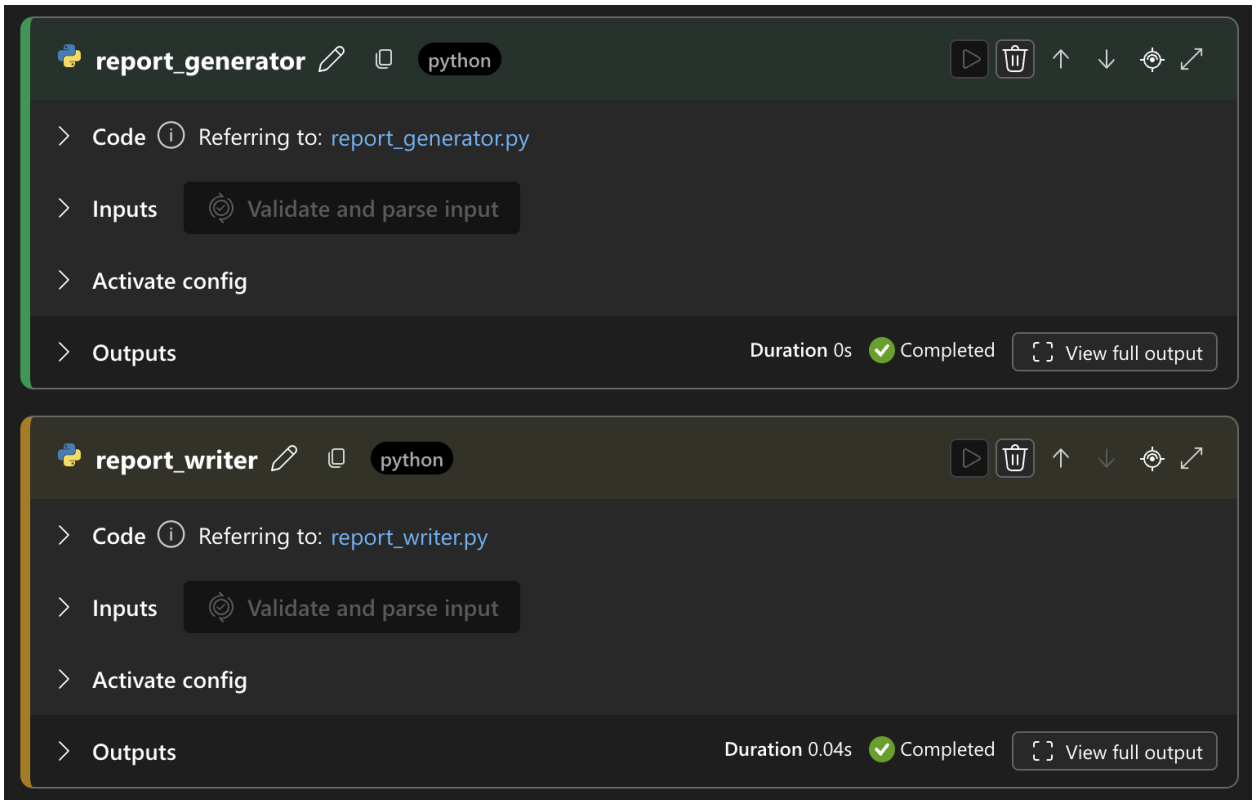

Fig. 10: report_generator and report_writer node config

4. Configuration, Governance, and Reusability

The project follows best practices for maintainability and governance as outlined in the Phase 2 guide:

- Centralized configuration via a config.env file for endpoints, Agent IDs, and credentials.
- Clear separation of concerns between:
  - Workflow orchestration (Hub Project).
  - Domain intelligence and standards (Foundry Project).
- Explicit grounding of interpretation logic in internal capability guidance using RAG, reducing the risk of hallucinated or inconsistent recommendations.

This design ensures that the workflow can be reused across additional processes with minimal changes (e.g., swapping datasets, specification limits, or RAG documents).

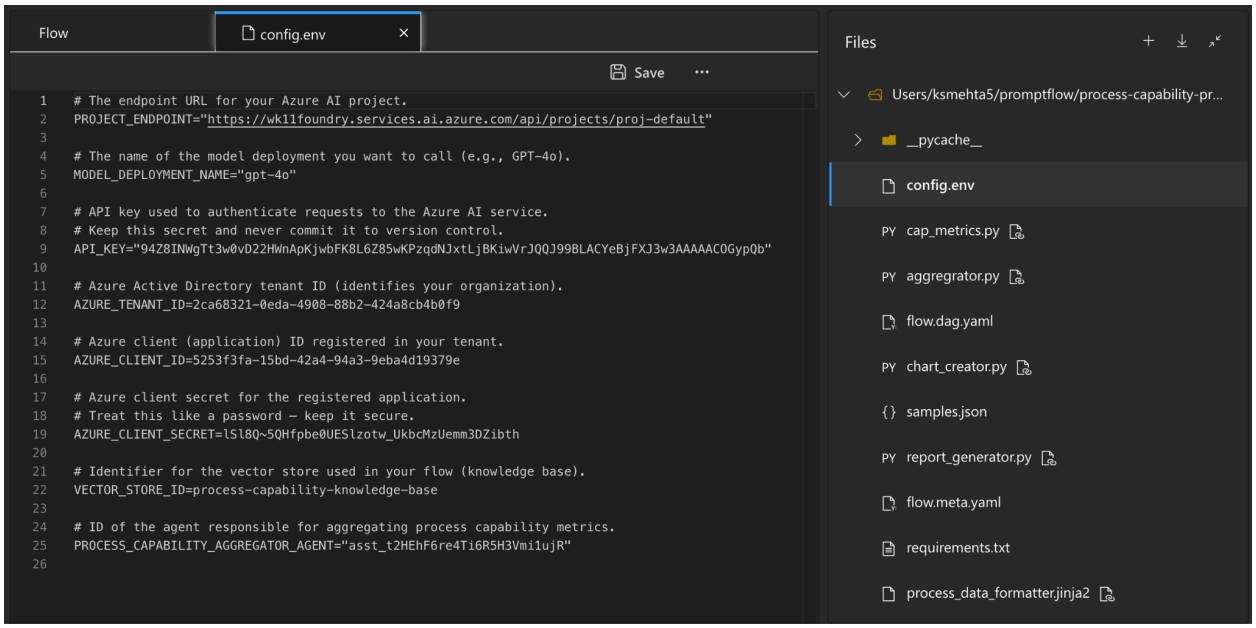

Fig. 11: Config.env file to manage key and endpoint access while keeping secrets discreet

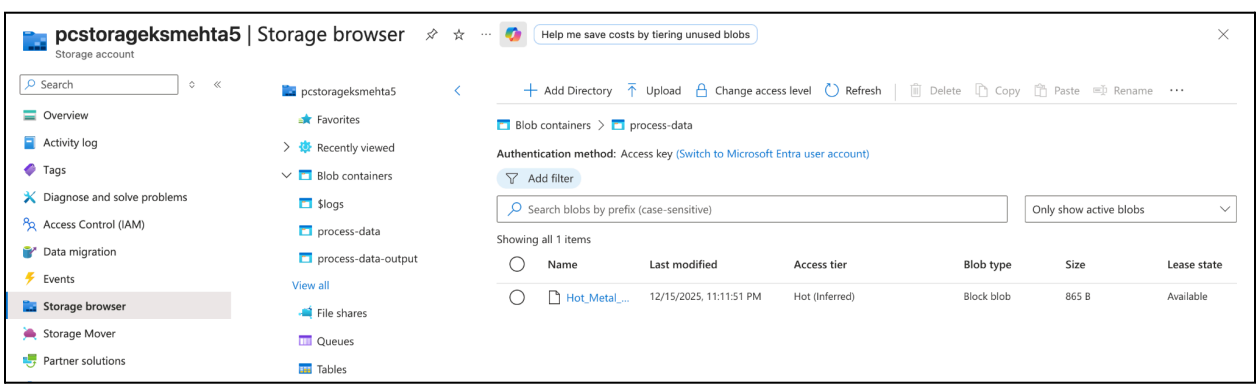

Fig. 12: Azure Blob Storage container consisting of the process data files

5. Management-Facing Deliverables

The final output of the workflow, i.e., the Aggregator Agent's synthesized report, directly supports the pilot requirement for a management-facing deliverable.

The Aggregator output:

- Clearly separates process stability from process capability.
- Warns when capability indices should not be trusted due to instability.
- Provides concise, actionable recommendations suitable for VP-level stakeholders.

This output can be reused directly in:

- Written executive summaries.
- Slide decks for quality and operations leadership.
- Automated notifications or reports in a future production deployment.

- **Interpretation:**

The process is **statistically stable but incapable** of meeting specifications (35–65). Key issues include excessive variation relative to the spec range and off-centered production, contributing to high defect risks. Additional centering toward the target mean (50) is critical.

Stability vs. Capability

- **Context Alignment:** Stable processes typically support valid capability analysis. Here, statistical stability exists, but the capability values clearly reflect that the process cannot meet specification requirements.
- **Conflict Resolution:** There is no significant conflict; however, improvement efforts must prioritize reduction of variation and recentralizing the process mean.

Management Recommendations

1. **Immediate Investigations:**

- Address assignable causes linked to variability spikes (sample range ~95–120), focusing on operator actions and external influences.
- Examine conditions leading to high moving ranges; inspect tooling setup, procedural consistency, and environmental factors.

Fig. 13: Snapshot of the final process capability report generated by the Agentic AI system

6. Project Reflections

6.1 Architecture & Design

If I had to explain the architecture to a non-technical manager, I would describe it as a decision pipeline: raw process data goes in, specialized steps analyze stability and capability in parallel, and a final expert step produces a clear recommendation. Each component does one job well, which makes the system easier to understand and trust.

A major design decision was choosing where logic should live. I intentionally kept numerical calculations and chart generation in Python and MCP tools, because they need to be deterministic and auditable. I used LLMs where flexibility and reasoning add value, such as data normalization, interpretation, and narrative synthesis. This separation reduced the risk of hallucinated numbers while still benefiting from AI-driven insight.

6.2 Agentic Patterns

The three agent patterns behaved very differently in practice. The on-the-fly MCP agents were the most predictable and reliable, behaving like reusable utilities for standardized analytics. The multimodal agent was effective at interpreting visual patterns in charts, but sensitive to prompt wording, reinforcing the need for tight constraints. The persistent RAG-backed Aggregator Agent was where agentic AI clearly outperformed traditional scripting, particularly in reconciling stability, capability, and recommended actions in a consistent, standards-aligned narrative. This project clarified that agentic AI excels at interpretation and communication, while traditional tools remain better for precise computation.

6.3 Reliability & Trust

There were cases where outputs felt slightly overconfident, especially with borderline or noisy data. This highlighted that trust depends less on correctness and more on how clearly uncertainty is communicated. To improve reliability, I would strengthen prompts to require explicit caveats, expand RAG content with edge cases, and add simple checks to prevent capability interpretation when stability is not confirmed.

6.4 Scalability & Reuse

Scaling this approach across multiple processes would not require changing the workflow structure. The workflow design and Aggregator Agent pattern are reusable; what changes are the datasets, specifications, and sometimes the reference documents. This reinforced the value of separating process-specific inputs from reusable reasoning components.

6.5 Application Beyond This Capstone

I see this pattern being valuable anywhere data-driven decisions require interpretation, not just metrics, such as operational diagnostics, financial analysis, or risk reviews. The first pilot I would propose elsewhere would focus on explanation rather than automation, because this project showed that trust is built through clarity before control.